

ControlPlane.ai

Real-Time, Configurable AI Oversight for the Enterprise

Detailed Business Proposal — Round 2

Accenture Innovation Challenge 2026

Team: Innovators

Jay Chandwani | IIT Bombay, Economics

1. Executive Summary

Enterprises no longer run a single AI system — they run many, simultaneously, each with a different risk profile. A customer-facing chatbot, an internal copilot, and a regulated decision-support tool all carry the same underlying risks — hallucination, cost inefficiency, bias, and data leakage — but a one-size-fits-all oversight approach is either too slow for real-time interactions or too permissive for high-stakes decisions.

ControlPlane.ai is a configurable, real-time oversight layer that sits between any AI model and its downstream consumer. It scores every response across performance, cost, and responsibility dimensions, and applies a use-case-specific policy — allowing, editing, blocking, or escalating responses — rather than a single fixed rule set. The architecture is model-agnostic (it inspects only inputs and outputs), and the policy layer is fully configurable per use case, geography, and risk appetite, directly addressing the reality that hard-coded rules age quickly as regulation and use cases evolve.

This document expands the Round 1 concept into a complete solution design, a working prototype scope, a business case grounded in the brief's reference parameters, a phased enterprise rollout roadmap, and an honest accounting of risks and mitigations.

2. Problem Framing

Today, AI oversight is either absent, applied uniformly across very different use cases (and therefore wrong for most of them), or bolted on after the fact through periodic audits. Three specific complexities make this harder than a simple monitoring layer can solve:

- Risk categories overlap in practice. A fabricated detail about a person can simultaneously be a hallucination and a privacy concern — clean categorization is harder than it first appears.
- There is often no reliable, real-time ground truth to check a claim against. The same knowledge gaps that cause an AI to hallucinate also make automated verification difficult.
- The real design problem is calibration, not detection. Over-flagging causes alert fatigue and teaches users to bypass warnings; under-flagging creates real liability. Any credible system must be deliberately tuned toward this tradeoff, not pretend to solve it away.

Reframed problem statement: enterprises need an oversight layer that adapts its sensitivity and response to the specific use case it is protecting, operates without reliable ground truth, and avoids the alert fatigue that causes oversight to be ignored.

3. Target Users

Round 2's reference parameters describe an enterprise operating multiple AI use cases at once. We define three concrete personas, each requiring a different configuration of the same underlying checker:

Persona	AI System Owned	Risk Tolerance	Latency Budget
Customer Support Lead	Customer-facing support chatbot	Low tolerance for reputational risk; moderate tolerance for minor factual slips	Real-time — seconds
Internal Knowledge Manager	Internal copilot for employees	Higher tolerance — mistakes are internally correctable	Real-time, less latency-sensitive

Persona	AI System Owned	Risk Tolerance	Latency Budget
Risk / Compliance Officer	Decision-support tool in a regulated workflow (e.g. credit, hiring)	Near-zero tolerance — legal/regulatory exposure	Can tolerate slower, review-style latency

Each persona receives a distinct policy configuration from the same underlying detection and decision engine — see Section 7.

4. Solution Design

4.1 Solution Overview

ControlPlane Checker is a configurable oversight layer that sits between any AI model and its downstream consumer, scoring every response and applying a use-case-specific policy rather than one fixed rule set. The same detection logic feeds different decision thresholds depending on which persona or use case is being protected — this configurability is the core design principle that makes the solution generalizable rather than bespoke per deployment.

4.2 Detection Techniques

Technique	What It Catches	Rationale
Rule-based PII / entity detection (regex + lightweight NER)	Privacy leaks, sensitive data in output	Deterministic and fast — avoids the ground-truth problem entirely
AI-as-judge pattern	Hallucination risk; overlapping bias/privacy issues	A second, cheaper model call scores the response for consistency and plausibility against the original context — a relative judgment rather than an absolute correctness check
Self-consistency sampling (extension)	Confidently-wrong detection	Sample the same prompt multiple times and flag high disagreement; proposed as a future extension beyond the core prototype

Embedding/statistical anomaly detection and retrieval verification against a document corpus are noted as future extensions — they require historical data or a source corpus that is heavier to stand up within the prototype timeframe.

4.3 Decision Logic — Confidence Scoring & Tiered Response

Each response receives a composite risk score per dimension, computed from the PII detector (severity flag) and the AI-as-judge score (a 0–100 confidence rating of how well-supported the response is). Tiering thresholds are read from a per-persona policy configuration, not hard-coded:

Tier	Trigger	Action
Allow	Below all configured thresholds	Pass through; log silently
Edit / Flag	Medium-confidence breach	Inline warning or redaction; user proceeds
Block	High-confidence, high-severity breach (e.g. PII detected)	Response withheld; fallback message shown
Escalate	Low-confidence / ambiguous	Logged for asynchronous human review; response still served

This directly resolves the alert-fatigue-versus-liability tradeoff: the threshold changes per use case, not the underlying logic. A regulated decision-support tool uses tighter thresholds (more escalation); a customer chatbot uses looser thresholds (fewer interruptions, since mistakes are more recoverable).

4.4 Architecture

The checker sits as inline middleware between the AI model and the end user — intercepting the response before delivery, but after generation. Latency is protected through a fast-path / slow-path split:

- Fast-path (inline, synchronous): cheap, deterministic checks — PII regex, policy threshold lookup. Sub-100ms overhead.
- Slow-path (async, non-blocking): the AI-as-judge call carries real latency (hundreds of ms to seconds). The response is served immediately with a provisional status; the judge score is computed in parallel, and flagged cases are escalated for review rather than retroactively withdrawn.
- Configurable exception: for zero-tolerance use cases, the policy can specify synchronous judge scoring — accepting higher latency in exchange for stricter pre-delivery review.

```
User Request -> AI Model -> [ControlPlane Checker] -> Response to User
|
|-- Fast-path (inline, blocking)
|   - PII / entity regex
|   - Policy config lookup
|
|-- Slow-path (async, non-blocking)
|   - AI-as-judge scoring
|   - Logged for escalation if needed
```

4.5 Governance — The Policy Layer

A versioned policy configuration (JSON/YAML) is defined per use case, specifying detection thresholds, scoring mode (sync/async), and audit retention. This is what allows the same underlying engine to behave differently for a customer chatbot versus a regulated decision tool, and what keeps the system adaptable as regulation evolves — rule sets are swappable, not hard-coded.

```
use_case: customer_support_chatbot
jurisdiction: IN
thresholds:
  pii_detection: block
  hallucination_score:
    escalate_below: 40
    edit_below: 65
    allow_above: 65
scoring_mode: async
audit_retention_days: 90
```

```
use_case: credit_decision_support
jurisdiction: IN
thresholds:
  pii_detection: block
  hallucination_score:
    escalate_below: 70
    edit_below: 85
    allow_above: 85
```

```
scoring_mode: sync
audit_retention_days: 2555
```

Every decision (allow / edit / block / escalate) is logged with an input hash, detection scores, the policy version used, and the final action — sufficient to reconstruct why a decision was made for both internal review and regulatory inquiry.

5. Generalization for Broader Adoption

The architecture and detection mechanism generalize across models, use cases, and industries by design:

- Model-agnostic: the checker only touches input/output, never model internals, so it works identically regardless of the underlying foundation model or vendor.
- Use-case-agnostic via the policy layer: new use cases require a new configuration file, not new detection logic.
- Industry-agnostic at the architecture level: the same tiered allow/edit/block/escalate pattern applies across domains; only thresholds and applicable regulation change.

This generalization has honest limits, which the design accounts for rather than obscures:

- Regulatory content is jurisdiction-specific. A detector tuned for one regulatory regime will not automatically cover another — the architecture generalizes, but rule sets need a pluggable “regulatory rule pack” loaded per deployment.
- The AI-as-judge technique has a cost/latency profile that scales with usage. At tens of thousands of interactions per week, judging every single response is not economically realistic — caching, sampling, and a cheaper first-pass filter are necessary and are discussed in Section 8.
- Severity judgment is domain-specific and cannot be fully automated. What counts as “high severity” differs between a hiring tool and a support chatbot — this is precisely why severity thresholds live in the configurable policy layer rather than in the detection code.

6. Business Case & Impact

Using the brief’s reference parameters — an enterprise running tens of thousands of interactions per week across a customer support assistant, an internal knowledge assistant, and a decision-support tool, drawing on a mix of well-governed and loosely governed data sources — the case for ControlPlane.ai rests on three points:

6.1 Cost of the Status Quo

Enterprises today discover AI failures reactively, through customer complaints, legal exposure, or periodic audits weeks after the fact. Published estimates of production LLM hallucination rates (commonly cited in the low single-digit percentages) imply that at tens of thousands of weekly interactions, hundreds of risky responses reach users each week without any live check.

6.2 Value Delivered

- Risk reduction, not risk elimination: the system will not catch every issue — no oversight layer can, given the ground-truth constraint — but it shifts the distribution of when problems are caught, from after the user acted to before or immediately after.

- Reduced alert fatigue relative to a uniform policy: use-case-specific thresholds avoid drowning low-risk teams in false positives while giving high-risk use cases the scrutiny they need.
- Audit-readiness as a byproduct: the same audit trail generated for oversight doubles as compliance documentation, avoiding a separate cost center for regulated use cases.

6.3 Honest Framing on ROI

Quantifying a specific dollar figure is not credible without real deployment data, which the brief explicitly does not provide. Impact is instead framed as a risk-posture shift and an operational cost-avoidance category — a more defensible position for a skeptical stakeholder than a fabricated percentage.

7. Phased Roadmap

This roadmap describes a realistic enterprise rollout of ControlPlane.ai once adopted — distinct from the team's own prototype build timeline for this competition.

Phase	Scope	Illustrative Duration
Phase 1 — Pilot	Deploy on one use case (lowest-risk, e.g. internal copilot); rule-based PII detection + AI-as-judge scoring; manual threshold tuning from early false positive/negative review	6–8 weeks
Phase 2 — Multi-use-case rollout	Extend policy layer to customer-facing and regulated use cases with distinct configs; introduce async/sync mode split; build audit trail and reporting dashboard	2–3 months
Phase 3 — Feedback loop & scale	Incorporate flagged/overridden case feedback into threshold tuning; extend regulatory rule packs per geography; introduce sampling/caching to control judge-call cost at scale	Ongoing

8. Risks & Mitigations

Risk	Mitigation
The AI-as-judge itself can be wrong or hallucinate its judgment	Never treat judge output as ground truth; combine with deterministic rule-based checks wherever a check is possible (e.g. PII), and report judge-based scores with explicit uncertainty
Alert fatigue if thresholds are mis-tuned at launch	Start conservative (bias toward escalation) in Phase 1; tune down over time using logged override data — never start loose and tighten later
Cost/latency blowup at scale from judging every response	Introduce sampling (judge a representative subset for lower-risk use cases) and caching for repeated or similar queries
Regulatory drift as law changes	Rule packs are versioned and swappable via the policy configuration, never hard-coded into detection logic
Over-reliance on the checker reducing human vigilance	Audit trail surfaces confidence and uncertainty explicitly rather than a binary safe/unsafe label, keeping humans in the loop by design for escalated cases
Enterprises consume models via API, limiting internal inspection	The design assumes input/output-only checking from the outset — a deliberate architectural choice, not a limitation discovered late

9. Assumptions

The following assumptions, adapted from the brief's illustrative reference parameters, ground the design and business case:

- An enterprise operates multiple AI use cases concurrently — a customer support assistant, an internal knowledge assistant, and a decision-support tool — each with different latency and risk tolerance.
- Combined interaction volume across these use cases is on the order of tens of thousands per week.
- Underlying data sources feeding these AI systems are a mix of well-governed and loosely governed internal sources.
- Primary reference jurisdiction: India, with data protection considerations informed by the DPDP Act; the policy layer's rule-pack design is intended to extend to other jurisdictions (e.g. GDPR) without architectural change.

10. Conclusion

ControlPlane.ai reframes AI oversight from a uniform, after-the-fact discovery process into a live, configurable layer that adapts to the specific use case it protects. By separating detection (what is checked) from policy (how strictly it is enforced), the design generalizes across models, industries, and regulatory environments while remaining honest about where full automation is not possible — particularly around ground-truth verification and domain-specific severity judgment. The working prototype accompanying this proposal demonstrates the core mechanism: detection, tiered decision logic, and configurable, auditable governance, operating end-to-end on simulated data across the three defined personas.